CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir

Habib University, Fall 2025

Classes and Objects

Classes , Objects , Constructor and Destructor , Access modifier , Manager Function , Overloaded constructors

C++ Classes

In C++, the concept of class has been generalized in an object-oriented sense:

classes are types representing groups of similar instances each instance has certain fields that define it (instance variables) instances also have functions that can be applied to them (represented as function fields) -- called ***methods*** the programmer can limit access to parts of the class (to only those functions that need to know about the internals)

General Class Definition in C++

```
class ClassName {
  access_modifier1:
    field11_definition;
    field12_definition;
    …
  access_modifer2:
    field21_definition;
    field22_definition;
  …
};
```

ClassName is a new type (just as if declared as struct with typedef in C)

Possible access modifiers:

```
public
private
protected
```

Fields following a modifier are of that access until the next modifier is indicated

Fields can be variables (as in C structures) or functions

A Motivating Example

You declare a simple structure:

```
class Robot {
  float locX;
  float locY;
  float facing;
};

int main()
{

   Robot r1;
}
```

A Simple Class

```
class Robot {
public:
    float getX() { return locX; }
    float getY() { return locY; }
    float getFacing() { return facing; }
    void setFacing(float f) { facing = f; }
    void setLocation(float x, float y);
private:
    float locX;
    float locY;
    float facing;
};

int main()
{
    int x ;      //creating a variable of type int

    Robot r1;    // creating an object of type Robot
}
```

The members inside a class can be written in any order.

Doesn't matter what comes first (in general).

Interpretation of Classes as a **Data Type**

- The programming language provides you with primitive data types, like an **int** or a **double**.
- You can think of classes as a **user-defined data type**.
- Because we're trying to **simulate the real world**, primitive data types don't complete the job effectively.
    - Hence, having classes are useful.

Accessing an Object's Members

In order to access an object's members (attributes or methods), use the dot operator.

```
Robot r1;          // creating an object

r1.facing;         // accessing attributes (ERROR!!!! Because it is private)
                   //will be discussed later

r1.setFacing(2);   // accessing methods
```

```
class Robot {
public:
   float getX() { return locX; }
   float getY() { return locY; }
   float getFacing() { return facing; }
   void setFacing(float f) { facing = f; }
   void setLocation(float x, float y);
private:
   float locX;
   float locY;
   float facing;
};

int main()
{
   int x ;     //creating a variable of type int

   Robot r1,r2;   // creating an object of type Robot
   r1.setLocation(0,0); // calling meothod for r1 object

}
```

Accessing an Object's Members

In the last example, if I printed **r2**'s position attributes, **after** the function call was made, what would be their values?

Unaffected.

Reason: The data of different objects are independent from one another.

Accessing an Object's Members

- Understand that an object's data is **independent** of other objects' data.
- Each robot, for e.g. has their own color.
  - Spray painting (changing the color) of any robot **will not** change the color of all the robots of that class.
- Each employee will have their own unique ID.
- A class dictates what attributes its objects will have, but the attributes' values for those objects are independent from one another.

Robot Category

Ground Orange Robot

Ground Green Robot

Employee Category

Female Supervisor

Male Supervisor

Calling Member Functions

When you call a member function of an object, like so,

```
R1.setLocation(0,0);
```

The object, **r1**, is **implicitly** passed to **moveRobot()**.

```
setLocation(&r1)    // compiler does this for us.
```

And the attributes that you accessed within **setLocation()** were of this **implicit object** (**r1** in this case).

Calling Member Functions

That's why the method knows that it needs to modify **r1**'s attributes.

Concept of **this** pointer.

Talk more later.

Access Specifiers

Public, Private and Protected

Member Access

Each member in a class has a property called an **access level**.

This determines who can access that member.

C++ has three different access levels.

- Public
- Private
- Protected *(will talk about during inheritance)*

Public Members

- Accessible anywhere outside the class.

- *Accessible within the same class (by the member functions).*
- *Accessible within other classes.*
- *Accessible within other functions in the global scope (including **main()**).*

- But always, these public members will be accessed through an object.

Private Members (Default Access Level in Classes)

Members of a class are **private by default**.

That is, if you don't specify any access specifier, members will be considered private.

Private members are members in a class that are only accessible by other members of the **same class**.

`private` Access Modifier

Fields marked as `private` can only be accessed by functions that are part of that class.

In the Robot class, locX, locY, and facing are private float fields, these fields can only be accessed by functions that are in class Robot (getX, getY, getFacing, setFacing, setLocation).

Example:

```
void useRobot() {
  Robot r1;
  r1.locX = -5; // Error
```

`public` Access Modifier

Fields marked as `public` can be accessed by anyone

In the Robot class, the methods getX, getY, etc. are public

these functions can be called by anyone

Example:

```
void useRobot() {
  Robot r1;
  r1.setLocation(-5,-5); // Legal to call
```